Software Development Engineer Intern - Take-Home Assessment

Objective

Build two small services that process order updates and maintain the current net position for each trading symbol. The exercise evaluates streaming data processing, inter-service communication, state management, validation, testing, API design, and documentation.

A simple, correct, and well-tested solution is preferred over unnecessary infrastructure or abstraction.

Input

We will provide `order_updates.csv` with the following columns:

`event_id,symbol,transaction_type,quantity`

Example rows:

```
event_id,symbol,transaction_type,quantity
evt-0001,RELIANCE,BUY,90
evt-0002,TCS,SELL,75
```

An accepted event has this logical structure:

```
{
  "event_id": "evt-0001",
  "symbol": "RELIANCE",
  "transaction_type": "BUY",
  "quantity": 90
}
```

Event Contract

- `event_id` must be a non-empty string and uniquely identifies an event.
- `symbol` must be a non-empty string. Preserve its supplied case and value.
- `transaction_type` must be exactly BUY or SELL.
- `quantity` must be a positive integer.
- The first valid event received for an `event_id` wins. Ignore later events with the same ID, even if another field differs.
- Invalid rows must be logged with a clear reason and skipped without stopping subsequent processing.

Services To Build

The two services must run as separate processes and communicate over a defined interface.

1. Order Update Service

- Read the CSV incrementally, one row at a time. Do not load the entire file into memory.
- Validate each row and convert it into an order event.
- Send valid events to the Position Maintaining Service in CSV order.
- Emit no more than 50 events per second. Exact sub-millisecond timing is not expected; a clear and configurable throttle is sufficient.
- Log accepted, rejected, and successfully sent events at an appropriate level.
- Log when input processing is complete.

2. Position Maintaining Service

- Receive events from the Order Update Service.
- Maintain the net position for each symbol in memory.

- BUY increases the position and SELL decreases it.
- Keep accepted `event_id` values in memory and ignore duplicate events.
- Provide `GET /position`, returning the current position for every symbol seen in an accepted event, including symbols whose net position is zero.
- Keep the endpoint available while events are being processed.

Example response:

```
{
  "RELIANCE": 90,
  "TCS": -75
}
```

JSON key order is not important. Negative and zero positions are valid.

Inter-Service Communication

You may use HTTP, gRPC, Redis Streams or Pub/Sub, ZeroMQ, custom TCP, or another mechanism you can justify. External infrastructure is not required; HTTP between the two services is an acceptable solution.

Document:

- why you selected the mechanism;
- the event payload or schema;
- how connection or delivery errors are surfaced; and
- any delivery limitations in your implementation.

Durable delivery, exactly-once broker guarantees, and recovery after a complete process restart are out of scope. In-memory idempotency state may reset when the Position Maintaining Service restarts.

Functional Requirements

- Both services must be independently runnable.
- The input file path, service address, and port must be configurable rather than fixed to one machine-specific path.
- Normal operation and errors must produce useful terminal logs.
- Malformed input must not crash either service.
- Position updates and API reads must remain correct if they occur concurrently.
- Persistence is explicitly out of scope.

Required Tests

Include automated tests for at least:

- BUY and SELL position calculations.
- Multiple symbols, including negative and zero net positions.
- Duplicate `event_id` handling.
- Invalid transaction types.
- Zero, negative, non-integer, and blank quantities.
- Blank event IDs and symbols.
- Continuing with later rows after an invalid row.
- The `GET /position` response.

An end-to-end test covering both running services is encouraged but not mandatory. Avoid flaky tests that depend on highly precise timing.

Non-Goals

You do not need to implement:

- database persistence;

- authentication or authorization;
- a frontend or dashboard;
- distributed deployment or orchestration;
- cloud infrastructure;
- exactly-once delivery across restarts; or
- production monitoring.

Docker and Docker Compose are optional.

Required Deliverables

Publish the completed solution in a public GitHub repository containing:

1. Source code for both services.
2. Automated tests.
3. A dependency file appropriate for the selected language.
4. A clear `README.md` containing:
    - architecture and communication choices, with a brief reason for each;
    - step-by-step setup and run instructions;
    - commands to run the tests;
    - configuration options;
    - example API usage and response; and
    - known limitations or trade-offs.

The supplied CSV may be included because it is synthetic assessment data.

Important

- You may use any programming language. Python and Rust are preferred but are not mandatory.
- Use Git throughout the exercise and maintain a clear, meaningful commit history.
- State any use of AI-assisted tools and be prepared to explain all submitted code and design decisions.
- We value correctness, tests, readability, and clear reasoning more than the number of frameworks or tools used.

Evaluation Criteria

Area	Weight
Core functionality	30%
Validation and idempotency	20%
Service design and communication	15%
Automated tests	15%
Code quality	10%
Documentation and reproducibility	10%